Smart Pantry Manager — Project Report

Table of Contents

1. Introduction
2. System Design
   - Screen Flow
   - Database Model
   - How the Parts Connect
3. Key Code Snippets
   - A. Validate Pantry Details on the Server
   - B. Use a Prepared SQL Statement When Adding an Item
   - C. Run Network Calls Off the Main Thread
   - D. Compare Recipe Quantities Using Compatible Units
4. Challenges and Solutions
5. Conclusion and Reflection
6. Reference List

---

1. Introduction

Smart Pantry Manager is an Android app that helps people keep track of the food and ingredients they have at home. It is aimed at anyone who wants to organise their pantry and find recipe ideas based on what they already have.

Users can add, edit, and remove pantry items. Each item has a name, a quantity, a unit of measurement, and an optional expiry date. The app also shows a list of South African recipes and highlights which ones can be made using the ingredients currently in the pantry. A settings screen lets the user turn expiry alerts on or off.

The Android app is written in Java. It sends requests to a Java REST API, which reads and writes data in a PostgreSQL database.

---

2. System Design

Screen Flow

The app has a bottom navigation bar that stays visible across the main screens. The ingredient form opens when the user adds a new ingredient or chooses to edit an existing one. Tapping a recipe opens its detail screen.

```
flowchart TD
    P[Pantry] -->|Add ingredient| F[Add / Edit Ingredient]
    P -->|Tap item, then Edit| F
    P -->|Tap item, then Delete| P
```

```
    P -->|Bottom navigation| R[Recipe Collection]
    P -->|Bottom navigation| S[Settings]
    R -->|Suggested Recipes| G[Suggested Recipes]
    R -->|Select recipe| D[Recipe Details]
    G -->|Select recipe| D
    R -->|Bottom navigation| P
    R -->|Bottom navigation| S
    S -->|Bottom navigation| P
    S -->|Bottom navigation| R
```

Database Model

The database has three tables. `pantry_items` stores what the user currently has in stock. `recipes` holds the recipe catalogue, including the required ingredients stored as a JSON array inside the same row. `app_settings` stores the expiry-alert preference and is limited to a single row.

Because the ingredients are stored as JSON inside the `recipes` table, there is no separate join table or foreign key linking recipes to pantry items.

```
erDiagram
    PANTRY_ITEMS {
        BIGINT id PK
        TEXT name
        NUMERIC quantity
        TEXT unit
        DATE expiry_date
        TIMESTAMPTZ updated_at
    }
    RECIPES {
        BIGINT id PK
        TEXT name UK
        TEXT steps
        JSONB ingredients
    }
    APP_SETTINGS {
        INTEGER id PK "fixed value 1"
        BOOLEAN expiry_alerts
    }
```

Key rules enforced by the schema: - `quantity` in `pantry_items` must be greater than zero. - `name` in `recipes` must be unique. - `ingredients` in `recipes` must be a valid JSON array, where each entry has a name, quantity, and unit. - `app_settings` can only ever have one row because its `id` column is constrained to the value `1`.

How the Parts Connect

The Android app sends HTTP requests to routes such as `/api/pantry` and `/api/recipes/suggested`. The API validates the request, runs a SQL query against PostgreSQL, and returns JSON. The Android emulator cannot use `localhost` to reach the host machine, so the app uses the address `10.0.2.2` on port `8080` instead.

---

3. Key Code Snippets

A. Validate Pantry Details on the Server

From `Server.java`:

```
if (name.isEmpty() || unit.isEmpty() || !Double.isFinite(quantity) || quantity <= 0) {
    throw new IllegalArgumentException(
            "name, positive quantity and unit are required");
}
```

This check runs at the API level, before anything is saved to the database. It rejects requests that are missing a name or unit, or that have an invalid quantity. This matters because requests can come from sources other than the Android form, so the server cannot rely on the app to validate first.

B. Use a Prepared SQL Statement When Adding an Item

From `Server.java`:

```
String sql = "INSERT INTO pantry_items(name,quantity,unit,expiry_date) VALUES(?,?,?,?)";
try (PreparedStatement statement = database.prepareStatement(
        sql, Statement.RETURN_GENERATED_KEYS)) {
    bindPantry(statement, ingredient);
    statement.executeUpdate();
}
```

The SQL structure is written separately from the values the user typed. This prevents SQL injection, because the database driver handles escaping. It also makes it straightforward to store a blank optional expiry date as a database `NULL` inside the shared `bindPantry` method.

C. Run Network Calls Off the Main Thread

From `Api.java`:

```
private static final ExecutorService REQUESTS = Executors.newSingleThreadExecutor();
private static final Handler MAIN_THREAD = new Handler(Looper.getMainLooper());

public static void request(String method, String path, JSONObject body, Done done) {
```

```
    REQUESTS.execute(() -> {
        // Send the HTTP request and read the response here.
        MAIN_THREAD.post(() -> done.call(result, error));
    });
}
```

Network requests can take time, and Android does not allow them on the main thread. The app runs them on a background worker thread. Once the response arrives, the result is posted back to the main thread so the screen can update safely. The code also retries the request up to three times if a connection error occurs.

D. Compare Recipe Quantities Using Compatible Units

From `Server.java`:

```
case "kg" -> new Measure(quantity * 1000, "mass");
case "g", "gram", "grams" -> new Measure(quantity, "mass");
case "l", "litre", "liter", "litres", "liters" ->
        new Measure(quantity * 1000, "volume");
case "ml", "millilitre", "milliliter" ->
        new Measure(quantity, "volume");
```

Before comparing amounts, the API converts kilograms to grams and litres to millilitres, then tags each value with a dimension (`mass`, `volume`, `piece`, etc.). This means a recipe that needs 1 kg of flour will match correctly against 1000 g in the pantry, and a mass amount can never accidentally satisfy a volume requirement.

---

4. Challenges and Solutions

Recipe matching needed more than ingredient names

Just because an ingredient name appears in the pantry does not mean a recipe can be made. The user might not have enough of it, or the units might not be compatible. The API checks every required ingredient, adds up all matching pantry entries, converts supported metric units, and only includes a recipe in the suggestions if every required amount is available.

The Android app and the API run in different environments

The Android emulator runs in a virtual machine and cannot use `localhost` to reach a service on the host computer. The app uses `10.0.2.2:8080` for this. Network calls also run on a background thread, and the retry logic means a slow or temporarily unavailable server does not freeze the screen.

Pantry data must be checked consistently

The ingredient form on the Android side checks for required fields, a positive quantity, and the expected date format before sending the request. The API then repeats the important checks and parses dates as real calendar dates. This two-layer approach means invalid data is rejected even if it arrives from a client other than the app.

---

5. Conclusion and Reflection

This project connects an Android app, a REST API, and a PostgreSQL database into one working application. It demonstrates moving data between these layers, validating user input on both sides, and using pantry stock to suggest recipes. It also handles South African recipes and common metric unit conversions.

Given more time, I would improve the expiry-alert feature so it actively identifies items nearing their expiry date and notifies the user. I would also add automated tests for recipe matching edge cases, and allow recipe data to be managed through the app rather than only through the database seed file.

---

6. Reference List

[1] Smart Pantry Manager, `README.md` — features, setup instructions, and API route list.

[2] Smart Pantry Manager, `backend/schema.sql` — PostgreSQL table definitions and constraints.

[3] Smart Pantry Manager, `backend/seed.sql` — starter recipe data.

[4] Smart Pantry Manager, `backend/src/main/java/za/co/smartpantry/api/Server.java` — REST endpoints, validation, database access, and recipe matching.

[5] Smart Pantry Manager, `app/src/main/java/za/co/smartpantry/Api.java` — Android HTTP client, background requests, and retry logic.

[6] Smart Pantry Manager, `app/src/main/java/za/co/smartpantry/BaseActivity.java`, `PantryActivity.java`, `IngredientFormActivity.java`, `RecipeCollectionActivity.java`, `SuggestionsActivity.java`, `RecipeDetailActivity.java`, and `SettingsActivity.java` — screen layout, navigation, and user interactions.